

Enigma encryption

Task overview

In that exercise You would have to implement enigma like machine decoding by brute forcing. Fact you know:

- the number of rotors (2),
- its encoding scheme
 - (letter $\oplus$ rotor + 1) for every forward passing rotor
 - (letter $\oplus$ reflector)
 - (letter $\oplus$ rotor + 1) for every backward passing rotor
- Rotor advancement scheme
 - every step the youngest rotor is advanced,
 - every 10th steps the second youngest,
 - every 100th step the third,
 - and so, on
- beginning of a message
 - “Hello, FPGA Hackathon! Your secret message is: ”

Send back the original message.

Theoretical background

Enigma was a cipher machine developed to encode diplomatic and military messages. In typical use, one operator typed the letters while a second one noted which lights illuminated. The rotor positions changed with every keypress, effectively changing the encoding key and making the cipher resistant to pattern-matching cryptanalysis. To read an incoming message, the recipient's rotor positions had to match the rotor positions used by the sender when encoding.

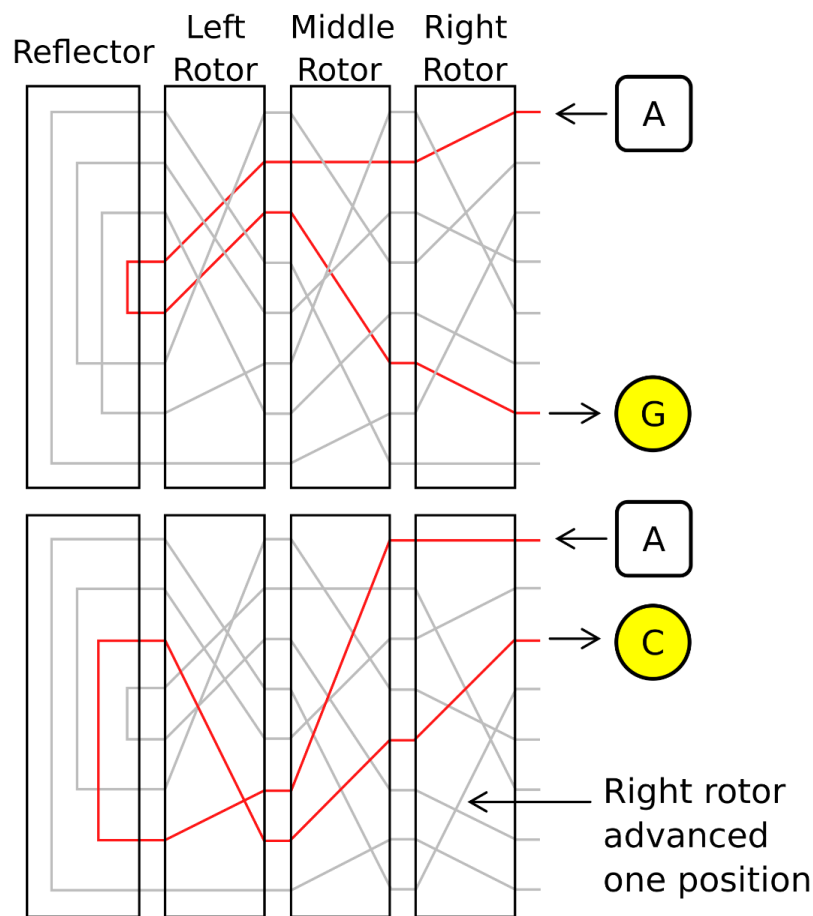


Image credit: [Wikimedia Commons](#). Licensed under [CC-BY-SA-2.5,2.0,1.0](#).

For this exercise, we decided to simplify the Enigma machine by making several simplifications.

All data, from the incoming message to the rotor positions, is represented as ASCII numbers to simplify the implementation. This drastically increases the complexity of the problem, since the number of possible combinations is:

$$N = N_{rotor\ position}^{N_{rotors}+1}$$

So even 3 rotors operating over ASCII 128 have a complexity like 5 rotors operating over the original 26 letters.

We eliminated the keyboard wiring and instead of the rotor wiring where some letters relate to others, we implemented such logic. Encoding pass:

$$f(x) = (x \oplus \omega) + 1$$

(1)

Decoding pass:

$$f^{-1}(x) = (x - 1) \oplus \omega$$

(2)

Where ω represents the current rotor position and $\oplus$ represents the XOR operation.

Unfortunately, due to our choice of XOR, we had to abandon the self-reciprocity of the original machine (the decoding and encoding passes differ). If we used equation (1) on the forward pass and equation (2) on the backward pass, then, due to the inherent involution of XOR, we would end up with a poorly encoded message.

Rotor advancement is done such that at every step the youngest rotor is advanced, every 10th steps the second youngest, every 100th step the third, and so on.

The encoding steps can be represented in pseudo-code as follows:

```
FUNCTION encode_letter(letter)

    // Forward pass-through rotors
    FOR EACH rotation IN rotors_positions
        letter = letter XOR rotation
        letter = (letter + 1) MOD 128
    END FOR

    // Reflector step
    letter = letter XOR reflector

    // Backward pass-through rotors in reverse order
    FOR EACH rotation IN reverse(rotors_positions)
        letter = letter XOR rotation
        letter = (letter + 1) MOD 128
    END FOR

    // Advance rotor positions
    advance_rotors()

    RETURN letter

END FUNCTION
```

Decoding steps can be represented as pseudo-code like this

```
FUNCTION decode_letter(letter)

    // Forward pass-through rotors

    FOR EACH rotation IN rotors_positions
        letter = (letter - 1) MOD 128
        letter = letter XOR rotation
    END FOR

    // Reflector step

    letter = letter XOR reflector

    // Backward pass-through rotors in reverse order

    FOR EACH rotation IN reverse(rotors_positions)
        letter = (letter - 1) MOD 128
        letter = letter XOR rotation
    END FOR

    // Advance rotor positions

    advance_rotors()

    RETURN letter

END FUNCTION
```

Goal

The goal of this exercise is to break the Enigma machine by brute-forcing all possible rotor and reflector combinations of the **2 rotors** enigma, given that the message starts with a predefined known prefix – **“Hello, FPGA Hackathon! Your secret message is: ”**. Solving it by other means is highly encouraged if we have left flaws in our encryption. If you feel ready for such a challenge, maybe go read some books about cryptography or something 😊. The FPGA should send back the decoded message, including the known prefix. Note that max message size is 1024 set by template parameter **“MAX_MSG_LENGTH”**

Provided module

You can edit anything, but we encourage you to limit it only to the *task_5.sv* file. If you wish, you may add other files for supportive submodules.

The module has the following inputs and outputs:

Table 1 List of inputs and outputs of the task_5 module.

Signal name	Signal type	Bit length
i_clk	Input	1 bit
i_valid	Input	1 bit
i_rst	Input	1 bit
i_data	Input	8 bits
i_last	Input	1 bit
i_first	Input	1 bit
o_data	Output	8 bits
o_valid	Output	1 bit
o_last	Output	1 bit

The reset is synchronous and active high.

Input and output interfaces

In one packet the task receives one sample of input data, 8 bits each. The last sample in a packet is indicated by signal *i_last*. The task should send one 8-bit sample and should indicate the last sample in the output packet by providing signal *o_last*.

Task receives data in the way shown in the figure below:

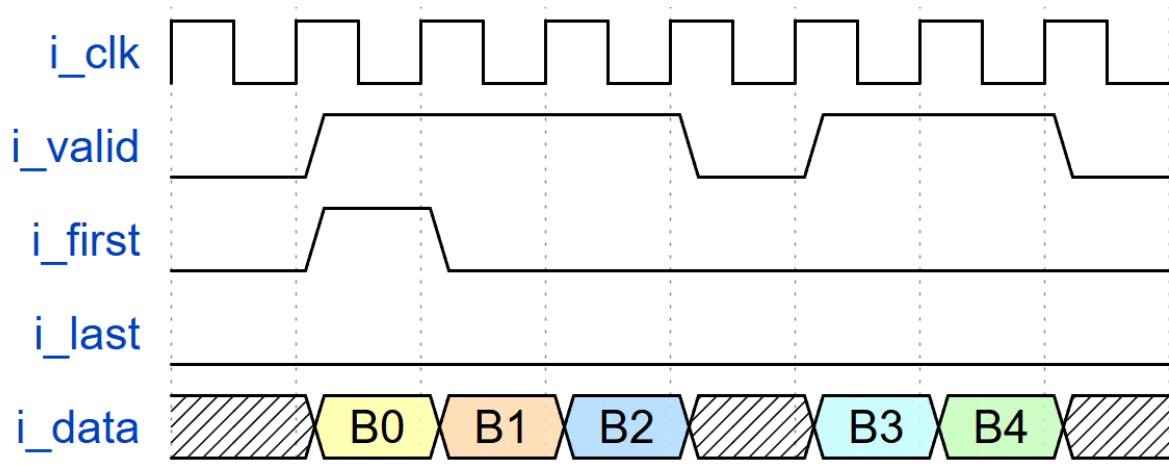


Figure 1 Input waveforms – the start of the packet.

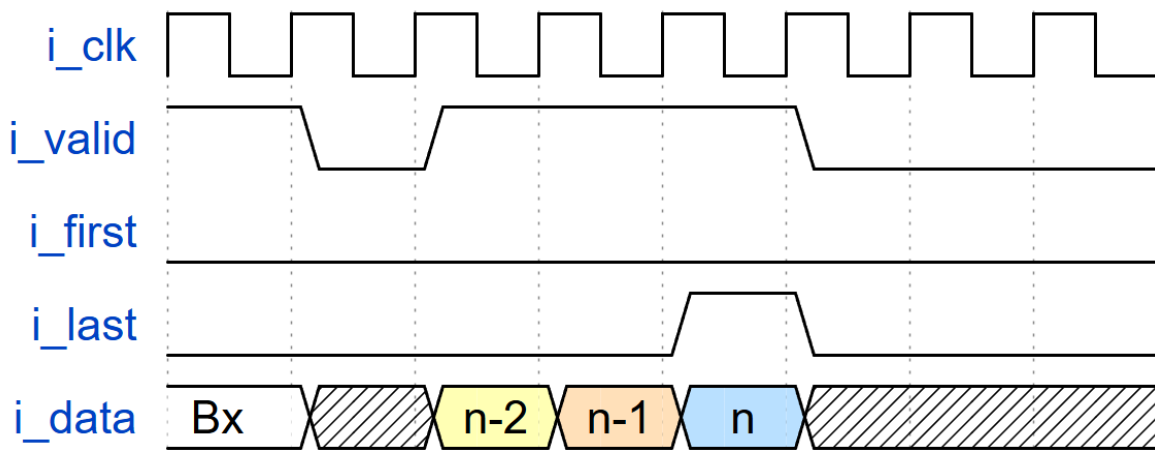


Figure 2 Input waveforms – the end of the packet.

To send data properly, the task must provide the *o_valid* signal that is synchronized with data on the *o_data* bus and *o_last* signal that is synchronized with the last sample in output packet.

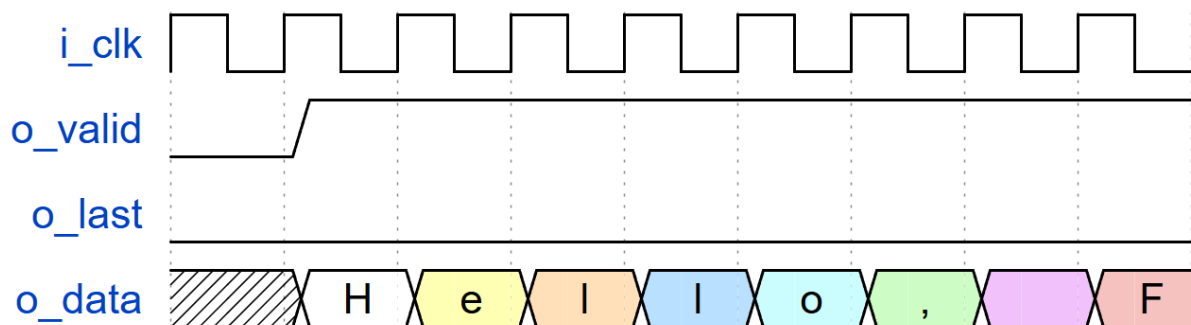


Figure 3 Output waveforms – the start of the packet.

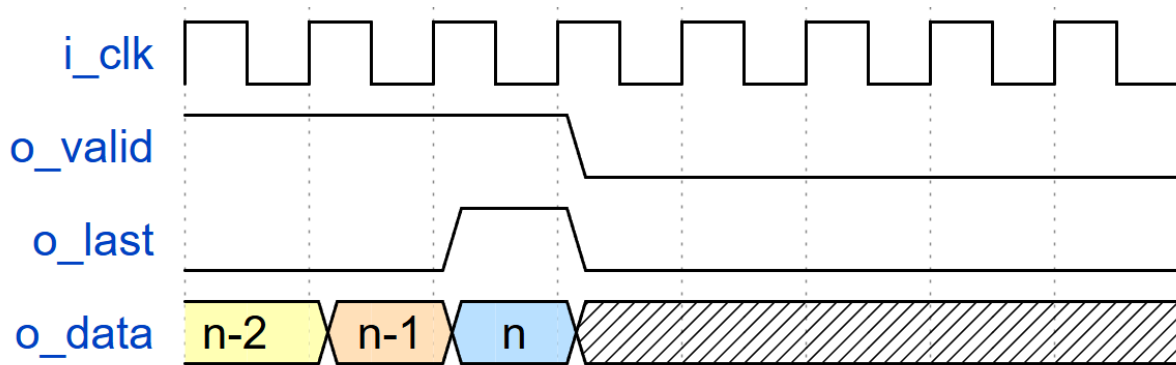


Figure 4 Output waveforms - the end of the packet.

Output data can be sent as a continuous stream, or it can have regular or irregular gaps. The only requirement is providing *o_valid* and *o_last* signals.

Evaluating the task

The task is evaluated by comparing the decoded message returned by the FPGA with the reference plaintext that was originally encoded by the testbench. A submission is considered correct only if every output byte matches the reference message, including the prefix known.

In Development mode, the testbench feeds your design a small, encoded message and reports per-sample mismatches together with the expected and observed values, so you can iterate functional correctness. In Evaluation mode, your design is run against a three, hidden encoded messages, and the score is based on the number of messages decoded fully and correctly. Each of the three randomly selected evaluation vector grants you 5 points, resulting in 15 points for a working implementation.

Additional points are awarded for reassures utilization up to 5 points provided that your latency is lower than 30 μ s cycles (for more information, check chapter 2.3.2 **Bonus Points** in the **FPGA_Hackathon_2026_Project_Guide** document).

